Computational Logic

Propositional Logic ct’d

Jakub Szymanik

Last week

- Completeness gives us a proof — and the proof it gives has 2^n pieces.
- Whether short proofs always exist is exactly NP = coNP. Nobody knows.
- So today we stop looking for proofs and start building solvers.

- Week 1 (14.09): Intro (try to read Logicomix after)
- Week 2 (21.09): How to Win a Million Dollars with Logic? Part 1:The language and what it means. (Study Sections 1.1, 1.3, 1.4.1-1.4.2 of the book)
- Week 3: (28.09) How to win a million dollars with logic? Part 2: Proof - and why it matches meaning. (Study 1.2, 1.4.3-1.4.4)
- Week 4: (05.10) How to win a million dollars with logic? Part 3: Normal forms and solvers. Lab, BY laptops (Study 1.5, 1.6)
- Homework 1.
- Week 5: (12.10) FO and non-computability (Study 2.1-2.2)
- Week 6: (19.10) FO and non-computability (Study 2.3-2.4)
- Week 7: (26.10) FO and non-computability (Study 2.5) + Trial intermediate exam.
- Homework 2.
- Week 8: (02.11) Expressiveness, Definability, and Generalized Quantifier Theory (Study 2.6-2.7, and the book chapter on Moodle)
- Week 9: (09.11) Modal Logic and Agency 1 (Study 5.2-5.3)
- Week 10: (16.11) Modal Logic and Agency 2 (Study 5.4-5.5)
- Week 11: (23.11) Modal Logic and Agency 3.
- Homework 3.
- Week 12: (30.11) Q&A + Trial final exam.

Conjunctive Normal Forms

Definition

Definition 1.42 A *literal* L is either an atom p or the negation of an atom $\neg p$. A formula C is in *conjunctive normal form* (CNF) if it is a conjunction of clauses, where each clause D is a disjunction of literals:

$$\begin{aligned} L &::= p \mid \neg p \\ D &::= L \mid L \vee D \\ C &::= D \mid D \wedge C. \end{aligned} \tag{1.6}$$

Conjunctive Normal Forms

Examples

$$(\neg q \vee p \vee r) \wedge (\neg p \vee r) \wedge q$$

$$(p \vee r) \wedge (\neg p \vee r) \wedge (p \vee \neg r)$$

Theorem. Every propositional formula can be rewritten as an equivalent CNF formula.

CONVERTING TO CNF — THE BOOK'S ALGORITHM, PP. 60–62

$(p \to q) \to (r \wedge s)$

one formula, four functions, nothing clever

IMPL_FREE	$\neg(\neg p \vee q) \vee (r \wedge s)$	*each → becomes ¬… ∨ …*
NNF	$(p \wedge \neg q) \vee (r \wedge s)$	*de Morgan inwards, ¬¬p collapses to p*
CNF / DISTR	$(p \vee r) \wedge (p \vee s) \wedge (\neg q \vee r) \wedge (\neg q \vee s)$	*one ∨ over two ∧ — this is where it costs*

WHY: DISTR PAIRS EVERY LEFT LITERAL WITH EVERY RIGHT ONE

	r	s
p	$p \vee r$	$p \vee s$
¬q	$\neg q \vee r$	$\neg q \vee s$

2 × 2 = 4 clauses

Stack n of these and the grid has 2^n cells. That is the whole of the blow-up.

function $\texttt{NNF}\,(\phi)$:

/* precondition: ϕ is implication free */

/* postcondition: $\texttt{NNF}\,(\phi)$ computes a NNF for ϕ */

begin function

case

ϕ is a literal: **return** ϕ

ϕ is $\neg\neg\phi_1$: **return** $\texttt{NNF}\,(\phi_1)$

ϕ is $\phi_1 \wedge \phi_2$: **return** $\texttt{NNF}\,(\phi_1) \wedge \texttt{NNF}\,(\phi_2)$

ϕ is $\phi_1 \vee \phi_2$: **return** $\texttt{NNF}\,(\phi_1) \vee \texttt{NNF}\,(\phi_2)$

ϕ is $\neg(\phi_1 \wedge \phi_2)$: **return** $\texttt{NNF}\,(\neg\phi_1) \vee \texttt{NNF}\,(\neg\phi_2)$

ϕ is $\neg(\phi_1 \vee \phi_2)$: **return** $\texttt{NNF}\,(\neg\phi_1) \wedge \texttt{NNF}\,(\neg\phi_2)$

end case

end function

function $\texttt{CNF}\,(\phi)$:

/* precondition: ϕ implication free and in NNF */

/* postcondition: $\texttt{CNF}\,(\phi)$ computes an equivalent CNF for ϕ */

begin function

case

ϕ is a literal: **return** ϕ

ϕ is $\phi_1 \wedge \phi_2$: **return** $\texttt{CNF}\,(\phi_1) \wedge \texttt{CNF}\,(\phi_2)$

ϕ is $\phi_1 \vee \phi_2$: **return** $\texttt{DISTR}\,(\texttt{CNF}\,(\phi_1), \texttt{CNF}\,(\phi_2))$

end case

end function

function DISTR (η_1, η_2):

/* precondition: η_1 and η_2 are in CNF */

/* postcondition: DISTR (η_1, η_2) computes a CNF for $\eta_1 \vee \eta_2$ */

begin function

case

η_1 is $\eta_{11} \wedge \eta_{12}$: **return** DISTR $(\eta_{11}, \eta_2) \wedge$ DISTR (η_{12}, η_2)

η_2 is $\eta_{21} \wedge \eta_{22}$: **return** DISTR $(\eta_1, \eta_{21}) \wedge$ DISTR (η_1, η_{22})

otherwise (= no conjunctions): **return** $\eta_1 \vee \eta_2$

end case

end function

Study examples on page 63

Exercises 1.5.8, 1.5.9 (at home)

What could be a computational benefit of having CNF formulas?

Lemma 1.43 *A disjunction of literals $L_1 \vee L_2 \vee \cdots \vee L_m$ is valid iff there are $1 \leq i, j \leq m$ such that L_i is $\neg L_j$.*

What if there were no exponential blow-up?



Proposition 1.45 *Let ϕ be a formula of propositional logic. Then ϕ is satisfiable iff $\neg\phi$ is not valid.*

Validity of a CNF — linear (Lemma 1.43);
Satisfiability of a CNF — NP-complete
Same formulas. Same syntax. One question is free; the other is the million-dollar one.
So where does the work go?

Tseitin, 1968 — you can have a linear CNF

Every formula has a CNF you can compute in linear time

- The trick: don't rewrite the formula — name it. A fresh variable per subformula, 3–4 clauses saying what each name means.
- So it is not equivalent to the original. It cannot be — it mentions variables the original never had.
- It is equisatisfiable: satisfiable exactly when the original is, and every model of it restricts to a model of the original.
- Which is why nothing here is free:
  - to decide satisfiability, you still have to solve the CNF — still NP-complete.
  - the linear validity test needs a CNF that is equivalent. This one is not.
  - The cost moved out of the translation and into the search.

Are there formulas for which SAT is easy?

Horn formulas

Definition

Definition 1.46 A *Horn formula* is a formula ϕ of propositional logic if it can be generated as an instance of H in this grammar:

$$\begin{aligned} P &::= \bot \mid \top \mid p \\ A &::= P \mid P \wedge A \\ C &::= A \to P \\ H &::= C \mid C \wedge H. \end{aligned} \tag{1.7}$$

We call each instance of C a *Horn clause*.

Horn formulas

Alternative definition

- Horn formulas are conjunctions of Horn clauses.
- Horn clause is an implication whose assumption A is a conjunction of propositions of type P and whose conclusion is also of type P, where
  - (P:= $\bot$| $\top$ | atom).

$$(p \wedge q \wedge s \rightarrow \bot) \wedge (q \wedge r \rightarrow p) \wedge (\top \rightarrow s)$$

Are those Horn formulas?

Quiz

$$(p_2 \wedge p_3 \wedge p_5 \rightarrow p_{13}) \wedge (\top \rightarrow p_5) \wedge (p_5 \wedge p_{11} \rightarrow \bot)$$

$$(p \wedge q \wedge s \rightarrow \neg p) \wedge (q \wedge r \rightarrow p) \wedge (p \wedge s \rightarrow s)$$

$$(p \wedge q \wedge s \rightarrow \bot) \wedge (\neg q \wedge r \rightarrow p) \wedge (\top \rightarrow s)$$

$$(p_2 \wedge p_3 \wedge p_5 \rightarrow p_{13} \wedge p_{27}) \wedge (\top \rightarrow p_5) \wedge (p_5 \wedge p_{11} \rightarrow \bot)$$

$$(p_2 \wedge p_3 \wedge p_5 \rightarrow p_{13} \wedge p_{27}) \wedge (\top \rightarrow p_5) \wedge (p_5 \wedge p_{11} \vee \bot)$$

Satisfiability-check

function HORN (ϕ):
/* precondition: ϕ is a Horn formula */
/* postcondition: HORN (ϕ) decides the satisfiability for ϕ */
begin function
    mark all occurrences of $\top$ in ϕ;
    while there is a conjunct $P_1 \wedge P_2 \wedge \cdots \wedge P_{k_i} \rightarrow P'$ of ϕ
        such that all P_j are marked but P' isn't **do**
        mark P'
    end while
    if $\bot$ is marked **then return** 'unsatisfiable' **else return** 'satisfiable'
end function

STUDY PROOF at HOME, p. 67:

Theorem 1.47 *The algorithm* HORN *is correct for the satisfiability decision problem of Horn formulas and has no more than* $n+1$ *cycles in its while-statement if* n *is the number of atoms in* ϕ. *In particular,* HORN *always terminates on correct input.*

Example

- (⊤→ q) ∧ (⊤ → s) ∧ (w →⊥) ∧ (p^q^s→v) ∧ (v→s) ∧ (⊤→r) ∧(r→p)
- Mark: q, s, r through (⊤→ q), (⊤ → s), (⊤→r)
- (⊤→ q) ∧ (⊤ → s) ∧ (w →⊥) ∧ (p^q^s→v) ∧ (v→s) ∧ (⊤→r) ∧(r→p)
- Mark: p through (r→p)
- (⊤→ q) ∧ (⊤ → s) ∧ (w →⊥) ∧ (p^q^s→v) ∧ (v→s) ∧ (⊤→r) ∧(r→p)
- Mark: v through (p^q^s→v)
- (⊤→ q) ∧ (⊤ → s) ∧ (w →⊥) ∧ (p^q^s→v) ∧ (v→s) ∧ (⊤→r) ∧(r→p)
- Return? – Satisfiable

Example

- $(\top \rightarrow q) \wedge (\top \rightarrow s) \wedge (w \rightarrow \bot) \wedge (p \wedge q \wedge s \rightarrow \bot) \wedge (v \rightarrow s) \wedge (\top \rightarrow r) \wedge (r \rightarrow p)$
- Mark: q, s, r through $(\top \rightarrow q)$, $(\top \rightarrow s)$, $(\top \rightarrow r)$
- $(\top \rightarrow q) \wedge (\top \rightarrow s) \wedge (w \rightarrow \bot) \wedge (p \wedge q \wedge s \rightarrow \bot) \wedge (v \rightarrow s) \wedge (\top \rightarrow r) \wedge (r \rightarrow p)$
- Mark: p through $(r \rightarrow p)$
- $(\top \rightarrow q) \wedge (\top \rightarrow s) \wedge (w \rightarrow \bot) \wedge (p \wedge q \wedge s \rightarrow \bot) \wedge (v \rightarrow s) \wedge (\top \rightarrow r) \wedge (r \rightarrow p)$
- Mark: $\bot$ through $(p \wedge q \wedge s \rightarrow \bot)$
- $(\top \rightarrow q) \wedge (\top \rightarrow s) \wedge (w \rightarrow \bot) \wedge (p \wedge q \wedge s \rightarrow \bot) \wedge (v \rightarrow s) \wedge (\top \rightarrow r) \wedge (r \rightarrow p)$
- Return? – Unsatisfiable

Why Horn?

- We have just proved the general problem is hard. The standard move in computer science is then: find the piece where it isn't.
- A Horn clause is a rule. $p_1 \wedge \ldots \wedge p_n \rightarrow q$ · "if all of these hold, then that holds" — the shape knowledge gets written in, and the whole of Prolog and Datalog.
- Satisfiability: linear. And you can tell by looking whether a formula qualifies.
- The boundary is exactly one positive literal per clause. Allow two, and it is NP-complete again.
- "A needs B" is Horn. "A needs one of B, C or D" is not — which is why real dependency resolvers ship a SAT solver.

Exercises 1.5.15

Section 1.6 of the book discusses how the marking algorithm could be extended beyond Horn formulas

- Nothing to hand in, nothing assessed. Get as far as you get.
- starters/ has the parser in Python, Java, and JavaScript — fill in evaluate, and you have something the checker will mark. python3 check.py classify --cmd "<however you run your program>"
- Then one of the two algorithms from the book. One is plenty.
- If you do not feel like programming, study Chapter 1.6 of the book instead.

The lab pack is on Moodle. One shared format so you can compare and swap code. A checker that runs your program, in whatever language you wrote it in.

1 · Get off the ground. Parser, then a brute-force truth-table evaluator. That evaluator is the oracle for everything else. starters/ has the parser done in three languages if you would rather skip it.

2 · Then one of the two algorithms from the book — the CNF pipeline (§1.5.2) or the Horn marking algorithm (§1.5.3). One is enough. Both are checked for you.

3 · Then make it explode. tests/blowup.in is the family from today, n = 1 to 11. Count the clauses your converter produces and find out whether your prediction was right. Then keep going until your machine complains.